

Calcolatori Elettronici

20 settembre 2023

Teoria

Esercizio 1 (12 punti): Progettare una rete sequenziale che riconosca la stringa in input 110 . La rete accetta in input i caratteri 0 e 1 e fornisce in output i valori "stringa riconosciuta"/"stringa non riconosciuta". Sintetizzare l'equazione di eccitazione di un flip flop a scelta e mostrarne il circuito equivalente.

Esercizio 2 (13 punti): Illustrare il funzionamento di un DMAC, corredando la spiegazione anche con uno schema circuitale dell'interfaccia tra DMAC e processore. Discutere i vantaggi dell'utilizzo di operazioni di trasferimento dati gestite da canale rispetto alla più tradizionale tecnica dell'I/O programmato.

Esercizio 3 (5 punti): data la funzione descritta dalla seguente tabella di verità:

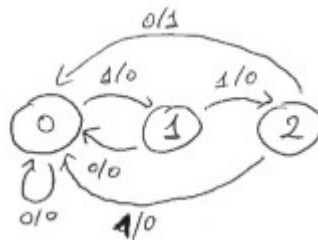
x	y	$f(x, y)$
0	0	0
0	1	1
1	0	1
1	1	0

esprimere la funzione minimizzata in forma canonica in somma di prodotti.

Soluzioni

Esercizio 1

Si propone la soluzione di una rete sequenziale che restituisce "stringa riconosciuta" ogni volta che viene identificata la stringa 110 in un input infinito. L'alfabeto di input, dalla traccia, è $I = \{0, 1\}$, mentre si sceglie di codificare l'output come $O = \{0, 1\}$ dove 0 indica "stringa non riconosciuta" e 1 "stringa riconosciuta". Utilizzando una macchina di Mealy, l'automa a stati finiti sarà:



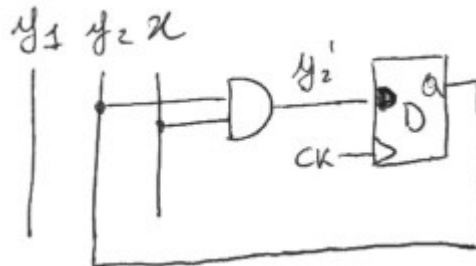
La tabella degli stati e transizioni corrispondenti è:

y_1	y_2	x	y_1'	y_2'	z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	1
1	0	1	0	0	0

Si sceglie di studiare l'equazione di eccitazione del flip flop del bit di stato y_1 . Disegnando la mappa di Karnaugh si ottiene:

	x	y_1	y_2		
		00	01	11	10
0		0	0	-	0
1		0	1	-	0

da cui $y_1' = y_2 x$. Il circuito equivalente sarà:



Esercizio 3

È immediato osservare che la funzione booleana data è lo xor, quindi si potrebbe scrivere $f = x \oplus y$. Tuttavia, viene richiesto di rappresentare la funzione in somma di prodotti. Dalla tabella di verità appare evidente che l'unica rappresentazione possibile è utilizzando direttamente i mintermini, ciò può essere anche verificato realizzando la mappa di Karnaugh:

	y	0	1
x	0	0	1
	1	1	0

La minimizzazione richiesta è quindi data da $f(x, y) = \bar{x}y + x\bar{y}$, che non è altro che la definizione di $x \oplus y$.

Progetto

Un processore `z64` controlla il sistema di irrigazione di un giardino. L'impianto utilizza una periferica `PROGRAMMATORE` che permette all'utente di specificare l'orario di accensione dell'impianto e la durata dell'irrigazione. Entrambi questi valori sono rappresentati come interi a 32 bit. Nel caso dell'orario di accensione, l'intero descrive il numero di secondi a partire dalla mezzanotte dopo i quali si intende avviare l'impianto. La durata di irrigazione è anch'essa espressa come numero di secondi.

La periferica `TIMER` invia una richiesta di interruzione al processore a cadenza di un secondo. Alla ricezione della richiesta di interruzione, il processore determina se si è raggiunto l'orario di accensione. In caso affermativo, lo `z64` programmerà la periferica `ELETTROVALVOLA` per attivare l'irrigazione. Al termine dell'intervallo richiesto dall'utente, la periferica `ELETTROVALVOLA` sarà programmata per arrestare l'irrigazione.

Una periferica `PLUVIOMETRO` permette allo `z64` di determinare se sono avvenute precipitazioni nel recente passato. Tale periferica sincrona fornisce il numero di *mm* d'acqua caduti nelle 24 ore precedenti alla lettura. L'intervallo di irrigazione richiesto dall'utente viene modificato in funzione della quantità d'acqua caduta, secondo la seguente logica:

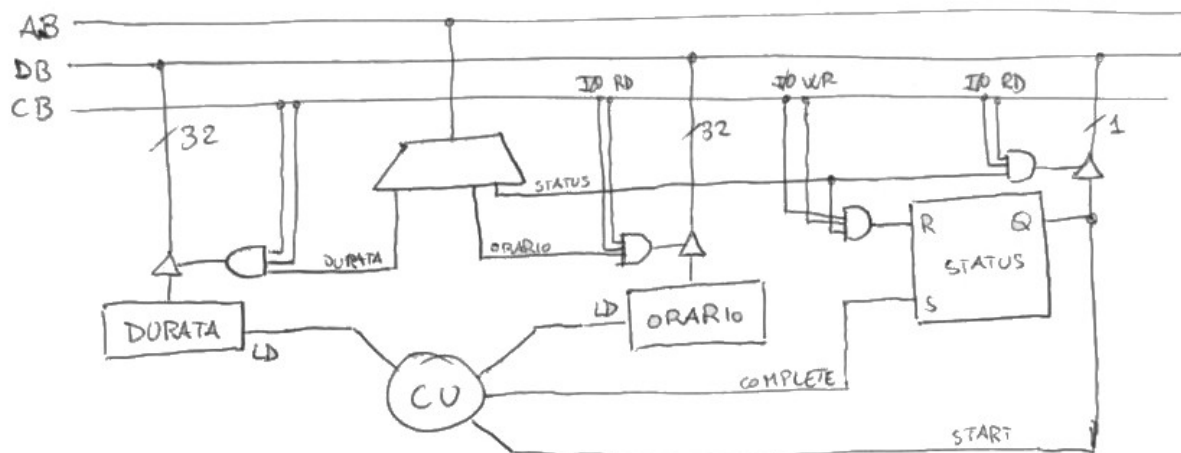
- viene lasciato inalterato se sono caduti meno di *48mm* d'acqua;
- viene dimezzato se sono caduti almeno *48mm* d'acqua ma non più di *96mm*;
- viene ridotto a un quarto se sono caduti più di *96mm* d'acqua.

Realizzare:

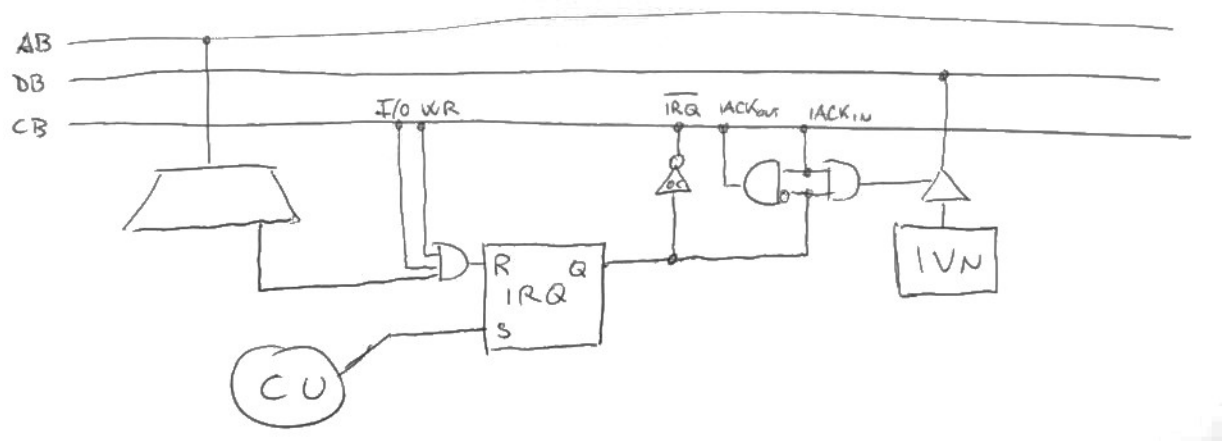
- Le interfacce delle periferiche `PROGRAMMATORE`, `TIMER` e `PLUVIOMETRO` (non è richiesta l'interfaccia di `ELETTROVALVOLA`);
- Tutto il software necessario al funzionamento del sistema, comprensivo di eventuali driver.

Soluzione

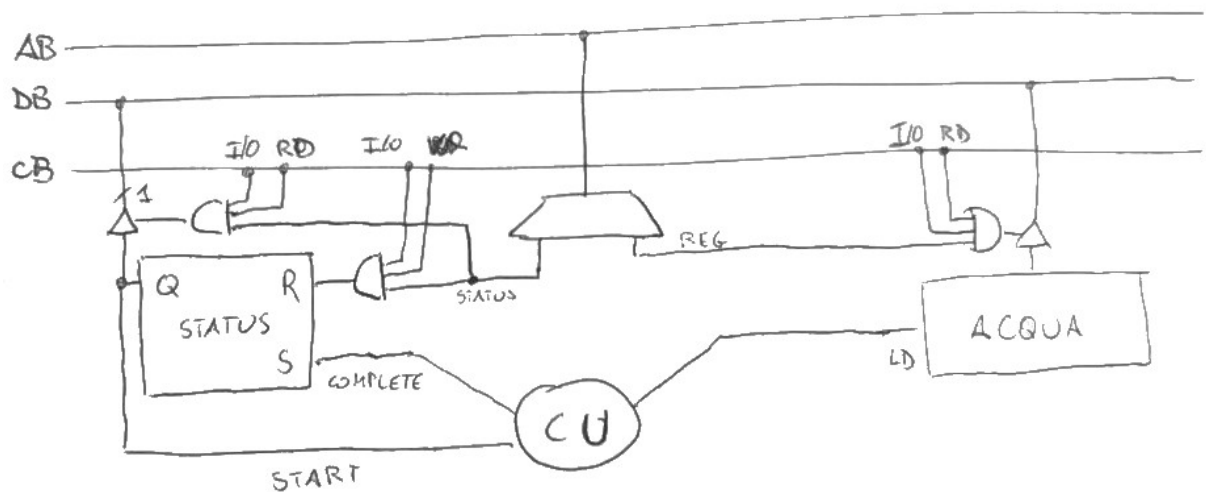
La periferica `PROGRAMMATORE` è una periferica sincrona di input. La sua peculiarità è quella di avere due registri di interfaccia per l'acquisizione dei dati. Come da protocollo classico per le periferiche sincrone, è necessario prevedere un flip flop di STATUS per notificare alla CU della periferica la necessità di scrivere i dati nei registri di interfaccia.



La periferica **TIMER** è una periferica asincrona. Dal momento che questa può sempre inviare richieste di interruzione, non è necessario avviarla esplicitamente.



La periferica **PLUVIOMETRO** è una periferica di input con un solo registro di interfaccia. Anch'essa necessita del flip flop di STATUS per richiedere alla CU la scrittura dei dati nel registro di interfaccia.



Il codice può essere il seguente.

```

1  .org 0x800
2  .data
3      .equ PR_DURATA, 0x1
4      .equ PR_ORARIO, 0x2
5      .equ PR_STATUS, 0x3
6      .equ TIMER_IRQ, 0x4
7      .equ PLU_STATUS, 0x5
8      .equ PLU_ACQUA, 0x6
9      .equ POCA_PIOGGIA, 48
10     .equ TANTA_PIOGGIA, 96
11     .equ UN_GIORNO, 86400 # secondi in un giorno
12     orologio: .long 0 # per semplicità assumo l'accensione a mezzanotte
13     attivo: .byte 0 # flag per indicare se l'impianto è attivo
14     orario_attivazione: .long 0
15     durata_attivazione: .long 0
16     orario_spegnimento: .long 0
17 .text
18 main:
19     sti
20     .loop:
21     call leggi_programmatore

```

```

22     hlt
23     call il_tempo_passa
24     call attiva_disattiva
25     jmp .loop
26
27 leggi_programmatore:
28     # Se l'impianto è attivo, non modifico le impostazioni
29     cmpb $1, attivo
30     jz .no_non_leggo
31     outb %al, $PR_STATUS
32     .bw_prog:
33     inb $PR_STATUS, %al
34     btb $0, %al
35     jnc .bw_prog
36     inl $PR_DURATA, %eax
37     movl %eax, durata_attivazione
38     inl $PR_ORARIO, %eax
39     movl %eax, orario_attivazione
40     .no_non_leggo:
41     ret
42
43 il_tempo_passa:
44     movq orologio, %rax
45     addq $1, %rax
46     cmpq $UN_GIORNO, %rax
47     jle .ancora_oggi
48     # È domani
49     xorq %rax, %rax
50     .ancora_oggi:
51     movq %rax, orologio
52     ret
53
54 attiva_disattiva:
55     cmpb $1, attivo
56     jz .controllo_spegnimento
57     # Se non attivo, confronta 'orologio' con 'orario_attivazione'
58     movq orologio, %rax
59     cmpq orario_attivazione, %rax
60     jle .end # Se orologio ≤ orario_attivazione, non faccio niente
61     movb $1, attivo
62     # Calcola la durata di attivazione
63     movq durata_attivazione, %rdi
64     call adjust_duration
65     addq orologio, %rax
66     movq %rax, orario_spegnimento
67     jmp .end
68     .controllo_spegnimento:
69     # Se 'attivo' è impostato, verifica se orologio > orario_spegnimento
70     movq orologio, %rax
71     cmpq orario_spegnimento, %rax
72     jle .end # Se orologio ≤ orario_spegnimento, non faccio niente
73     movb $0, attivo
74     .end:
75     ret
76
77 aggiusta_durata: # Primo argomento: durata richiesta
78     call quanta_pioggia
79     cmpq $POCA_PIOGGIA, %rax
80     jb .va_bene_cosi
81     cmpq $TANTA_PIOGGIA, %rax
82     jae .dividi_per_quattro
83     # Se è tra POCA_PIOGGIA e TANTA_PIOGGIA, dividi per 2
84     shrq $1, %rdi
85     movq %rdi, %rax
86     ret
87     .dividi_per_quattro:

```

```

88     shrq $2, %rdi
89     movq %rdi, %rax
90     ret
91     .va_bene_cosi:
92     movq %rdi, %rax
93     ret
94
95     quanta_pioggia:
96     outb %al, $PLU_STATUS
97     .bw_plu:
98     inb $PLU_STATUS, %al
99     btb $0, %al
100    jnc .bw_plu
101    inq $PLU_ACQUA, %rax
102    ret
103
104    .driver 0 # Timer
105    outb %al, $TIMER_IRQ
106    iret

```

Prova di laboratorio

Benvenuto nella biblioteca di Bugliano! La biblioteca ha bisogno del tuo aiuto per gestire il catalogo dei libri. I libri nella biblioteca dovranno essere gestiti tramite un software basato su una lista singolarmente connessa, in cui ogni nodo rappresenta un libro:

```

1 struct libro {
2     char titolo[100];
3     char autore[100];
4     unsigned anno;
5     struct libro *next;
6 };

```

La biblioteca di Bugliano ha ricevuto una donazione cospicua di nuovi libri. Il bibliotecario, grazie al software che realizzerete, dovrà inserirli nel catalogo, visualizzare il catalogo aggiornato, cercare un libro specifico per vedere se è disponibile e, purtroppo, eliminare alcuni libri vecchi per fare spazio ai nuovi.

Il software che dovete realizzare deve quindi prevedere le seguenti operazioni:

1. **Creazione e Inserimento:** scrivere una funzione `struct libro *crea_libro(char *titolo, char *autore, unsigned anno)` che inserisce un nuovo libro e restituisce il puntatore al nodo della lista. Successivamente, scrivere una funzione `struct libro *inserisci_libro(struct libro **head, struct libro *nuovo_libro)` che inserisce il nuovo libro in testa alla lista.
2. **Visualizzazione:** implementare una funzione `void visualizza_catalogo(struct libro *head)` che stampi il catalogo dei libri presenti nella biblioteca.
3. **Ricerca:** realizzare una funzione `struct libro *cerca_libro_per_titolo(struct libro *head, char *titolo)` che cerca nella lista un libro basandosi sul titolo e restituisce un puntatore al libro se trovato, altrimenti `NULL`.
4. **Eliminazione:** implementare una funzione `struct libro *elimina_libro(struct libro **head, char *titolo)` che elimina un libro dalla lista basandosi sul titolo e restituisce la testa aggiornata della lista.

Soluzione

Le funzionalità richieste altro non sono che operazioni tradizionali su liste singolarmente concatenate. Pertanto, la seguente è una possibile soluzione:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  struct libro {
6      char titolo[100];
7      char autore[100];
8      unsigned anno;
9      struct libro *next;
10 };
11
12 struct libro *crea_libro(char *titolo, char *autore, unsigned anno)
13 {
14     struct libro *new_libro = malloc(sizeof(struct libro));
15     if (new_libro == NULL) {
16         perror("Impossibile allocare memoria per il nuovo libro");
17         exit(1);
18     }
19     strncpy(new_libro->titolo, titolo, sizeof(new_libro->titolo) - 1);
20     strncpy(new_libro->autore, autore, sizeof(new_libro->autore) - 1);
21     new_libro->anno = anno;
22     new_libro->next = NULL;
23     return new_libro;
24 }
25
26
27 struct libro *inserisci_libro(struct libro **head, struct libro *nuovo_libro)
28 {
29     nuovo_libro->next = *head;
30     *head = nuovo_libro;
31     return *head;
32 }
33
34
35 void visualizza_catalogo(struct libro *head) {
36     printf("Catalogo della Biblioteca di Bugliano:\n");
37     while (head) {
38         printf("Titolo: %s\n", head->titolo);
39         printf("Autore: %s\n", head->autore);
40         printf("Anno: %u\n\n", head->anno);
41         head = head->next;
42     }
43 }
44
45
46 struct libro *cerca_libro_per_titolo(struct libro *head, char *titolo)
47 {
48     while (head) {
49         if (strcmp(head->titolo, titolo) == 0) {
50             return head;
51         }
52         head = head->next;
53     }
54     return NULL;
55 }
56
57
58 struct libro *elimina_libro(struct libro **head, char *titolo)
59 {
```

```
60     struct libro **current = head;
61
62     while (*current && strcmp((*current)→titolo, titolo) ≠ 0) {
63         current = &(*current)→next;
64     }
65
66     if (*current) {
67         struct libro *temp = *current;
68         *current = (*current)→next;
69         free(temp);
70     }
71
72     return *head;
73 }
74
```